SEAT NO. CT-23025

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

SECOND YEAR (Computer Science)

FALL SEMESTER EXAMINATIONS 2024

Time : 3 Hours | Batch 2023 | Dated : 10-DEC-24

Max Marks : 60

Data Structure Algorithms & Applications - CT-159

1. An adjacency list representation of a directed weighted graph is given below. Draw a directed graph by **interpreting** the given adjacency list representation. **Find** the shortest path from **source vertex "A"** to all other vertices in the given graph by applying Dijkstra's Algorithm. [CLO-01, Marks-05] .

Vertex	Neighbor : Edge Weight
A	→ E:1 → H:10 → NULL
B	→ C:2 → NULL
C	→ NULL
D	→ A:4 → H:1 → NULL
E	→ F:3 → NULL
F	→ B:1 → I:1 → C:3 → G:7 → NULL
G	→ NULL
H	→ E:5 → I:9 → NULL
I	→ J:2 → NULL
J	→ G:1 → NULL

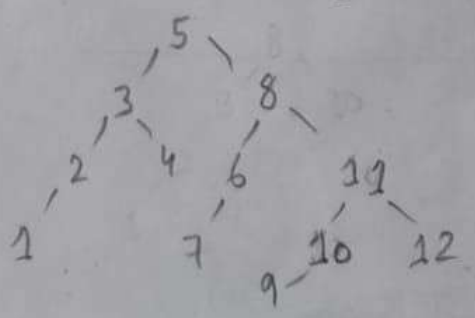
7
0, 5, ~~1~~, I, 1, 4, 8, 10,
5, 7

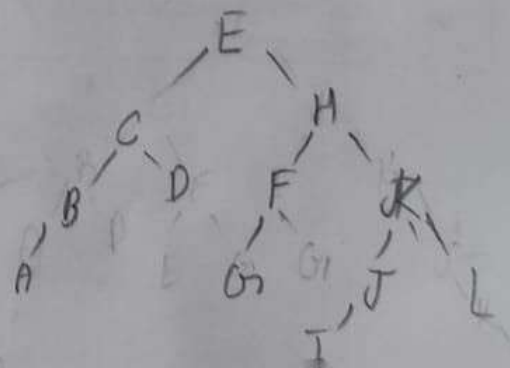
2. An Adjacency Matrix Representation of an undirected weighted graph is given below. Draw an undirected weighted graph by interpreting the given adjacency matrix representation. **Show** the minimum spanning tree obtained by applying i) Kruskal and ii) Prims Algorithms on the given graph. [CLO-01, Marks-05]

V	0	1	2	3	4	5	6	7	8
0	0	4	0	0	0	0	0	8	0
1	4	0	8	0	0	0	0	11	0
2	0	8	0	7	0	4	0	0	2
3	0	0	7	0	9	14	0	0	0
4	0	0	0	9	0	10	0	0	0
5	0	0	4	14	10	0	2	0	0
6	0	0	0	0	0	2	0	1	6
7	8	11	0	0	0	0	1	0	7
8	0	0	2	0	0	0	6	7	0

3 — 4
2 — 8
0 — 1 | 6 — 7
5
37
=> 37

3. Given an unsorted array [9, 4, 3, 8, 10, 2, 5], **translate/convert** it into a sorted array by applying in-place heap-sort algorithm. [CLO-01, Marks-05]
4. **Translate/ convert** the given array of elements [H, I, J, B, A, E, C, F, D, G, K, L] into an AVL tree by inserting elements one by one in an empty AVL tree. Apply proper rotations if required. [CLO-01, Marks-05]
5. Briefly **Explain** hashing, rehashing, and the issue of collision in hashing. [CLO-01, Marks-05]





6. You are given a hash table of size S=13. The hash table uses double hashing for collision resolution, with the following hash functions: $h_1(k) = k \% S$, $h_2(k) = 7 - (k \% 7)$. Given the following keys to insert into the hash table: 18, 41, 22, 44, 59, 32, 31, 73. For each key determine the position where the key will be inserted using the formula: $position = [h_1(k) + i * h_2(k)] \% S$, where $0 \leq i < S$ is the collision number. **Show** the final Hash Table. [CLO-01, Marks-05]
7. A browser history system can be designed using a doubly linked list, where each node stores a URL and has references to both the previous and next URLs. List the steps, pseudo-code, for methods to go forward, go backward, and add a new URL to the history? [CLO-02, Marks-05]
8. Given a Binary Search Tree (BST) that stores city names as keys and their populations as values. **List the steps, pseudo-code,** for a method that takes root node of the BST as input and prints **K** largest cities by population in descending order. Assume that K= last two digits of your roll number. The time complexity of your solution must be O(K). [CLO-02, Marks-05]
9. You are given a directed acyclic unweighted graph wherein each vertex represents a course code. **Discover** a linearized sorted order of these courses by applying the topological sort algorithm. Also, list the steps, pseudo-code, of the algorithm. [CLO-02, Marks-05]

Vertex	Neighbor
2	→ NULL
3	→ 8 →10 → NULL
5	→ 11 → NULL
7	→ 8 → 11 → NULL
8	→ 9 → NULL
9	→ NULL
10	→ NULL
11	→ 2 → 9 → 10 →NULL

10. **Analyze** the following pieces of programs, perform dry run for the given input **test cases**, and determine the output. [CLO-02, Marks-15]

Program	Test Case

a.
struct Customer {
string name;
bool isSenior;
};
void serveCustomers(queue<Customer>& customerQueue) {
while (!customerQueue.empty()) {
bool foundSenior = false;
int queueSize = customerQueue.size();
for (int i = 0; i < queueSize; ++i) {
Customer current = customerQueue.front();
customerQueue.pop();
if (!foundSenior && current.isSenior) {
cout << "Serving : " << current.name << endl;
foundSenior = true;
} else
customerQueue.push(current);
if (foundSenior)
	customerQueue = { {"Ahmed", false}, {"Bilal", true}, {"Kiran", false}, {"Elsa", true}, {"Evan", false} }

```
        break; }
    if (!foundSenior) {
        Customer current = customerQueue.front();
        customerQueue.pop();
        cout << "Serving : " << current.name << endl; } } Bilal
}
```

b.

```
int X(const vector<int>& arr, int k1, int k2) {
  priority_queue<int, vector<int>, greater<int>>
minHeap(arr.begin(), arr.end());
  for (int i = 1; i < k1; ++i)
      minHeap.pop();
  int k1 = minHeap.top();
  for (int i = k1; i < k2; ++i)
      minHeap.pop();
  int k2 = minHeap.top();
  int sum = 0;
  for (int i=0; i< arr.size(); i++) {
    if (arr[i] > k1 && arr[i] < k2)
      sum += arr[i];   }
  return sum;  }        0
```

arr[] = {7, 10, 4, 3, 20, 15},
k1 = 3, k2 = 6

c.

```
void X (int A, int B, const vector<vector<int>>& graph) {
      int n = graph.size();
      vector<bool> visited(n, false); T, T, T, F, F, F
      vector<int> parent(n, -1);  -1, 0, 1, -1, -1, -1
      queue<int> q;
      q.push(A);
      visited[A] = true;
      while (!q.empty()) {
        int current = q.front();
        q.pop();
        for (int n=0; n < graph[current].size(); n++) {
          if (!visited[n]) {
            visited[n] = true;
            parent[n] = current;
            q.push(n);
            if (n == B)
              break;
    }  }   }
}
```

A = 0, B = 5,
graph = { {1, 2}, {0, 3, 4},{0, 4},
{1, 5}, {1, 2, 5}, {3, 4} },

Note: The given graph includes in total 6 vertices.